```
# Import Required Libraries
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# ----------------------------------------
# Load Dataset
# ----------------------------------------

df = pd.read_csv("loan_data.csv")

# ----------------------------------------
# Remove Loan_ID Column
# ----------------------------------------

if "Loan_ID" in df.columns:
```

```
    df.drop("Loan_ID", axis=1, inplace=True)

# ----------------------------------------
# Handle Missing Numerical Values
# ----------------------------------------

numerical_columns = [
    "ApplicantIncome",
    "CoapplicantIncome",
    "LoanAmount",
    "Loan_Amount_Term",
    "Credit_History"
]

for column in numerical_columns:
    df[column] = df[column].fillna(df[column].mean())

# ----------------------------------------
# Handle Missing Categorical Values
# ----------------------------------------

categorical_columns = [
    "Gender",
    "Married",
    "Dependents",
    "Education",
```

```
    "Self_Employed",
    "Property_Area",
    "Loan_Status"
]

for column in categorical_columns:
    df[column] = df[column].fillna(df[column].mode()[0])

# ----------------------------------------
# Encode Categorical Values
# ----------------------------------------

encoder = LabelEncoder()

for column in categorical_columns:
    df[column] = encoder.fit_transform(df[column].astype(str))

# ----------------------------------------
# Separate Features and Target
# ----------------------------------------

X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]

# ----------------------------------------
# Split Dataset
```

```
# ----------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# ----------------------------------------
# Gradient Boosting Function
# ----------------------------------------

def xgboost(X_train, X_test, y_train, y_test):

    model = GradientBoostingClassifier(random_state=42)

    # Train Model
    model.fit(X_train, y_train)

    # Prediction
    y_pred = model.predict(X_test)

    # Accuracy
    train_accuracy = accuracy_score(
```

```
        y_train,
        model.predict(X_train)
    )

    test_accuracy = accuracy_score(
        y_test,
        y_pred
    )

    # Display Results
    print("=" * 50)
    print("Gradient Boosting Model Results")
    print("=" * 50)

    print("\nTraining Accuracy :", round(train_accuracy * 100, 2), "%")
    print("Testing Accuracy  :", round(test_accuracy * 100, 2), "%")

    print("\nConfusion Matrix")
    print(confusion_matrix(y_test, y_pred))

    print("\nClassification Report")
    print(classification_report(y_test, y_pred))

# ----------------------------------------
# Call Function
# ----------------------------------------
```

```
xgboost(X_train, X_test, y_train, y_test)
```